Problem Statement & Goal

- Chosen track is 2. The name of the project is Solar Energy Forecasting Using a Neural Network and Energy Scheduling Using Linear Programming
- **DISCLAIMER:** The project is not about the MPPT algorithm, which is utilized in all solar inverters.
- Suppose we have a household with a solar plant and an electricity storage (battery).
- The goal of the application is following: the user specifies the expected energy consumption at given times for the following time intervals (e.g. two days). Our product will forecast solar power in the following days using a recurrent neural network, based on current weather forecast, historical solar data, and current date. Afterwards, the application will schedule electricity grid consumption and battery charging plans which minimize the electricity bill the user has to pay. Also, the application will find all time slots with unused solar energy. In this problem, we will suppose flat electricity prices, so I only train one neural network for the solar output forecast (also majority of households use flat prices). We generally assume the electricity produced by the solar plant can be sold to the distributor for a flat price as well.
- Let's mention several heuristics to manage solar energy (already implemented in many smart homes):
  - Time-of-use charging: charge the battery during off-peak hours and discharge it during peak hours when the grid electricity price is high.
  - Priority-based consumption: prioritze critical loads, and if there's energy surplus, use it to charge the battery or any other load (such as water heater).
  - As mentioned in [1], solar inverters with AI already exist. However, we aim for a more modular, open and custom solution.

Problem Statement & Goal + Methodology Overview

- The problem with aforementioned strategies - not generalizable. For example, electricity prices may even be negative (in case of energy surplus). The solution is to develop a linear programming (LP) model and then find the optimal schedule at respective i time steps.
- Other approaches to the problem are:
  - Mixed-integer linear programming (MILP): necessary for complex binary "turn on/off" or integer constraints, in this project we assume everything is continuous. Definitely usable in future work;
  - Pure Reinforcement learning using NN: much faster to solve the prediction problem, however the problem does not require frequent energy schedules. Every 15 minutes is enough. Also does not guarantee optimal solution. In case of a different model (more powerful solar panels, higher capacity battery), the model needs to be retrained (in former case, we modify the constants only);
  - Genetic algorithms: does not guarantee optimal solution, but will probably be much faster than MILP if the problem has too many variables. Also does not require retraining.
- **Remaining Issues for Deployment:** Solar energy production and electricity prices forecasting. This will be the topic of Progress Check 2 and final report with a recurrent neural network.
- For Progress Check 1, we will focus on the LP model (our sandbox for overall testing).
- Project will be implemented in Python. For visualization (scheduled energy production and forecasted solar power production), I plan to program a webpage.

LP Modelling

We need to formulate the problem with the following variables (suppose timesteps $i \in \{1, ..., N\}$ sampled in the "zero-order hold" manner, length of the time step is T [h]). Also suppose "sanity-check" bounds (e.g. we cannot pull infinite amount of energy from a battery):

- $P_{\text{grid}}^{(i)} \in \left[P_{\text{grid}}^{\text{MIN}}, P_{\text{grid}}^{\text{MAX}}\right]$ [kW]: grid electricity consumption at time step i, negative in case the household sells energy to the grid, values are all *optimization variables*.
- $P_{\text{solaravail}}^{(i)} \in \mathbb{R}_0^+$ [kW]: solar power production at time step i, values are all *given constants* (in the final application predicted from the NN).
- $P_{\text{solar}}^{(i)} \in \left[0, P_{\text{solaravail}}^{(i)}\right]$ [kW]: used solar power in the household (converted by the solar inverter) at time step i, and the values are all *optimization variables*. We must also take into account the maximum power the solar inverter can produce. However, to reduce the number of variables, we can clip predicted NN data.
- $P_{\text{bat}}^{(i)} \in \left[P_{\text{bat}}^{\text{MIN}}, P_{\text{bat}}^{\text{MAX}}\right]$ [kW]: supplied power to the battery at time step i, values are all *optimization variables*. Positive value at time i means the battery discharges to supply electricity.
- $BC^{(i)} \in \left[0.1 \cdot BC^{\text{MAX}}, 0.9 \cdot BC^{\text{MAX}}\right]$: helper *optimization variables* which denote current battery charge, the charge is bounded in such a way the battery is not fully discharged neither charged, as we want to mitigate battery degradation.
- $P_{\text{load}}^{(i)} \in \mathbb{R}_0^+$ [kW]: specified household electricity demand at time step i, value are all *given constants*.
- η_{solar}: efficiency of DC solar power conversion to the AC household grid. This is a *constant*.
- PB $[\frac{\$}{\text{kWh}}]$: the price for which we buy 1 kWh of electricity. This is a *constant*.
- PS $[\frac{\$}{\text{kWh}}]$: the price for which we sell 1 kWh of electricity. This is a *constant*.

The problem assumes the battery can be charged either from the grid or the solar panel inverter. We also assume $PS < PB$, since usually the household sells for less than it buys. This makes the following function convex (needed for the LP solvers):

$$EC^{(i)} = \max\left\{PB \cdot T \cdot P_{\text{grid}}^{(i)}\left(\text{for } P_{\text{grid}}^{(i)} \geq 0\right), PS \cdot T \cdot P_{\text{grid}}^{(i)}\left(\text{for } P_{\text{grid}}^{(i)} < 0\right)\right\} \tag{1}$$

Let $EC^{(i)}$ be the energy cost at time step i (negative when electricity is sold, else otherwise).

LP Modelling Cont. & Finished

Finally, the optimization criterion and constraints:

$$\min \sum_{i=1}^{N} EC^{(i)} \tag{2}$$

$$P_{\text{load}}^{(i)} = P_{\text{grid}}^{(i)} + \eta_{\text{solar}} \cdot P_{\text{solar}}^{(i)} + P_{\text{bat}}^{(i)} \quad \text{Demand is satisfied.} \tag{3}$$

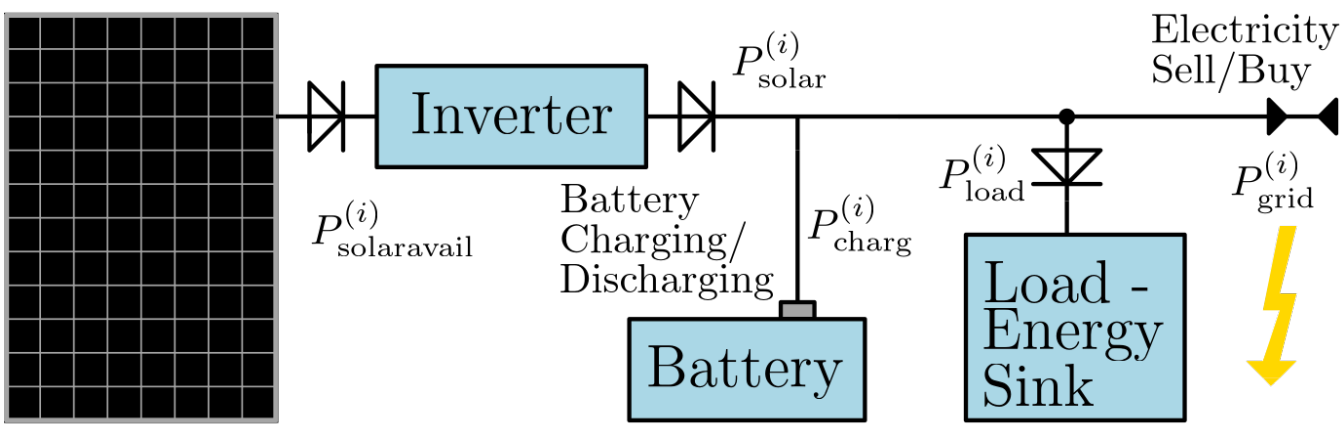
$$PB \cdot P_{\text{grid}}^{(i)} \leq EC^{(i)}, \quad PS \cdot P_{\text{grid}}^{(i)} \leq EC^{(i)} \quad \text{Energy cost is a max of two functions.} \tag{4}$$

$$BC^{(i+1)} = BC^{(i)} - P_{\text{bat}}^{(i)} \cdot T \quad \text{Battery charge state. We compute } BC^{(n+1)} \text{ as well.} \tag{5}$$

$$P_{\text{grid}}^{\text{MIN}} \leq P_{\text{grid}}^{(i)} \leq P_{\text{grid}}^{\text{MAX}}, \quad 0 \leq P_{\text{solar}}^{(i)} \leq P_{\text{solaravail}}^{(i)} \quad \text{Sanity checks.} \tag{6}$$

$$P_{\text{bat}}^{\text{MIN}} \leq P_{\text{bat}}^{(i)} \leq P_{\text{bat}}^{\text{MAX}}, \quad 0.1 \cdot BC^{\text{MAX}} \leq BC^{(i)} \leq 0.9 \cdot BC^{\text{MAX}} \quad \text{Sanity checks.} \tag{7}$$

$$BC^{(1)} = BC^{\text{INIT}} \quad \text{Initial battery charge.} \tag{8}$$



Schematic of the modelled system. The unidirectional energy flow is represented by a diode. A typical setup with an AC coupled battery is considered. Setups with a DC coupled battery connected directly to the inverter are commonly used as well. Charging from the AC grid is possible, as the inverter converts AC back to charge the DC coupled battery.

Implementation in Python

We can use the `scipy.optimize.linprog` function to solve the LP model (a wrapper of the HiGHS solver). The documentation states the problem needs to be formualted as:

$$\min c^T x \tag{9}$$

$$\text{s.t.}: A_{\text{ub}} x \leq b_{\text{ub}} \tag{10}$$

$$A_{\text{eq}} x = b_{\text{eq}} \tag{11}$$

$$lb \leq x \leq ub \tag{12}$$

$$\text{The optimization vector is formulated as: } x^T = \begin{pmatrix} EC^T & P_{\text{grid}}^T & P_{\text{solar}}^T & P_{\text{bat}}^T & BC^T \end{pmatrix} \tag{13}$$

The code below implements all the needed matrices and vectors.

```
ts = self.time_steps
# Criterion Vector (c)
self.c = np.zeros((5*ts, 1))
self.c[0:ts] = 1
# Nonequality Matrix (A_ub)
self.aub = np.zeros((2*ts, 5*ts))
# 1st eq
self.aub[0:ts, 0:ts]    = -np.eye(ts)
self.aub[0:ts, ts:2*ts] = self.grid_price_buy * np.eye(ts)
# 2nd eq
self.aub[ts:2*ts, 0:ts]    = -np.eye(ts)
self.aub[ts:2*ts, ts:2*ts] = self.grid_price_sell * np.eye(ts)
# Nonequality Vector (b_ub)
self.bub = np.zeros((2 * ts, 1))
# Equality Matrix (A_eq)
self.aeq = np.zeros((ts, 5*ts))
# 1st eq
self.aeq[0:ts, ts:2*ts]   = np.eye(ts)
self.aeq[0:ts, 2*ts:3*ts] = self.eff_solar * np.eye(ts)
self.aeq[0:ts, 3*ts:4*ts] = np.eye(ts)
```

Python LP Code


```
# 2nd eq
col_offset = 3*ts
self.aeq_2 = np.zeros((ts, 5*ts))
self.aeq_2[0:ts, col_offset:col_offset+ts] = self.dt * np.eye(ts)
self.aeq_2[0:ts, col_offset+ts:col_offset+2*ts] = np.eye(ts) - np.eye(ts, k=-1)
self.aeq = np.concatenate((self.aeq, self.aeq_2), axis=0)
# Equality Vector (b_eq)
self.beq = np.zeros((2 * ts, 1))
self.beq[0:ts, 0] = self.p_load.flatten()
self.beq[ts, 0] = self.initial_battery_capacity
# Lower Bound Vector (lb)
self.lb = 5*ts * [None]
self.lb[ts:2*ts]   = ts * [self.p_grid_bound[0]]
self.lb[2*ts:3*ts] = ts * [0]
self.lb[3*ts:4*ts] = ts * [self.p_bat_bound[0]]
self.lb[4*ts:5*ts] = ts * [0.1 * self.battery_capacity]
# Upper Bound Vector (ub)
self.ub = 5*ts * [None]
self.ub[ts:2*ts]   = ts * [self.p_grid_bound[1]]
self.ub[2*ts:3*ts] = self.p_solaravail.reshape(1, -1).tolist()[0]
self.ub[3*ts:4*ts] = ts * [self.p_bat_bound[1]]
self.ub[4*ts:5*ts] = ts * [0.9 * self.battery_capacity]

# ================= The solution ===================
bounds = list(zip(self.lb, self.ub))
result = scipy.optimize.linprog(self.c, A_ub=self.aub, b_ub=self.bub, A_eq=self.aeq, b_eq=self.beq, \
                                bounds=bounds, method='highs')
# The resulting vector is stored in result.x and the success is stored in result.success.
```

Results

I have downloaded daily power consumption data from [2] and manually created a `numpy` array with size 24 (one for each hour).

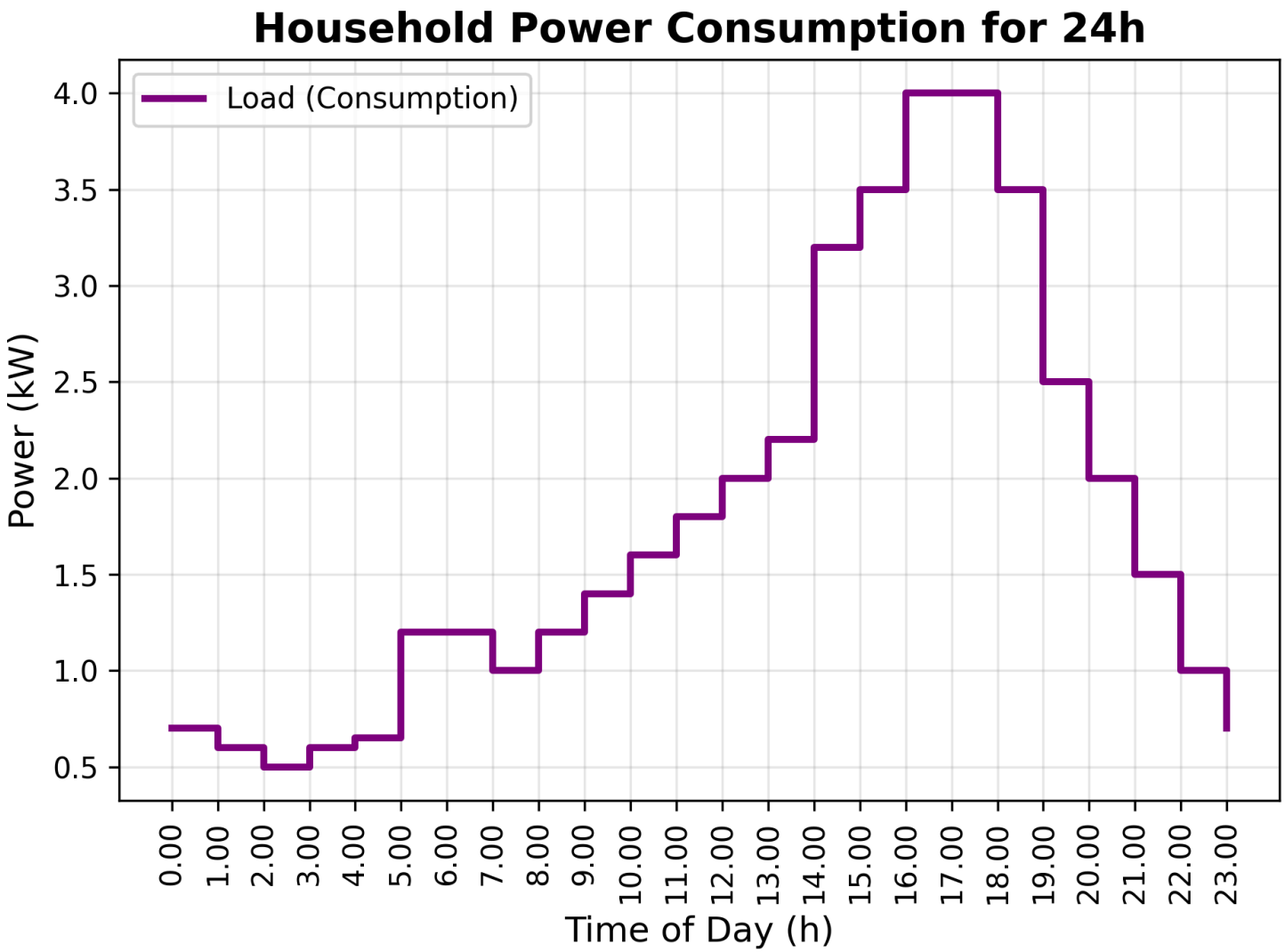


Figure 2: Household power consumption for 24h. It can be seen the consumption peaks in the morning and evening.

Results Cont.

I have also downloaded PV production curves from [3]. The summer day curve is shown in the figure below, as well as the results of the solver for two cases. The cases are described in the captions. The initial battery charge is 2 kWh. The battery capacity is 10 kWh, but we won't allow it to discharge below 1 kWh neither charge above 9 kWh. The solar inverter efficiency is 0.95. It must be mentioned that the two datasets are taken from different sources and are used for demonstration purposes only.

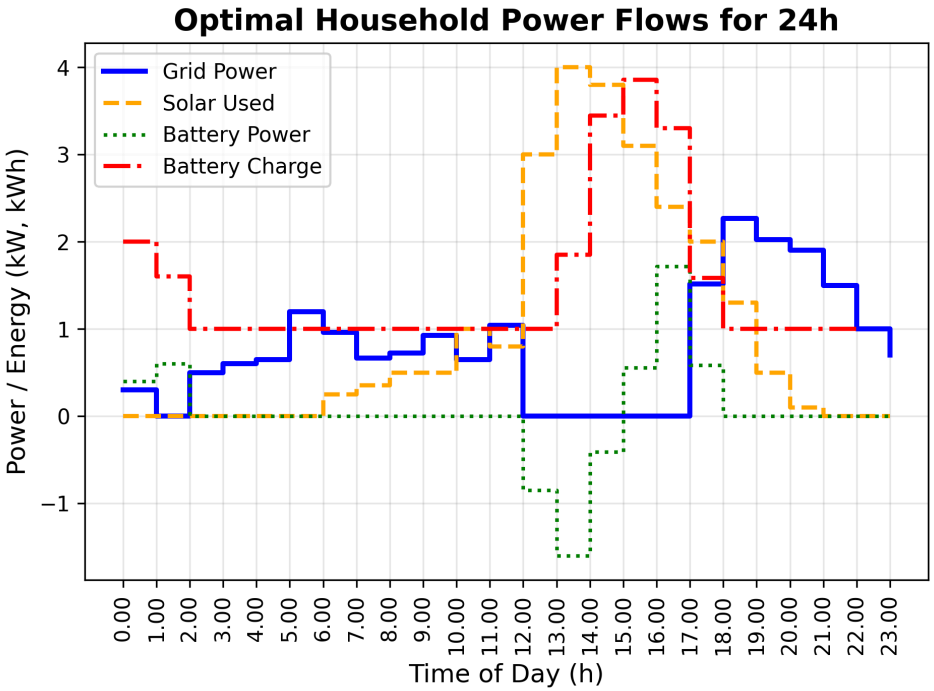


Figure 3: Buying price is 0.20 USD/kWh, selling price is 0.05 USD/kWh. We can see the battery is charged in the afternoon, so it can be discharged in the evening.

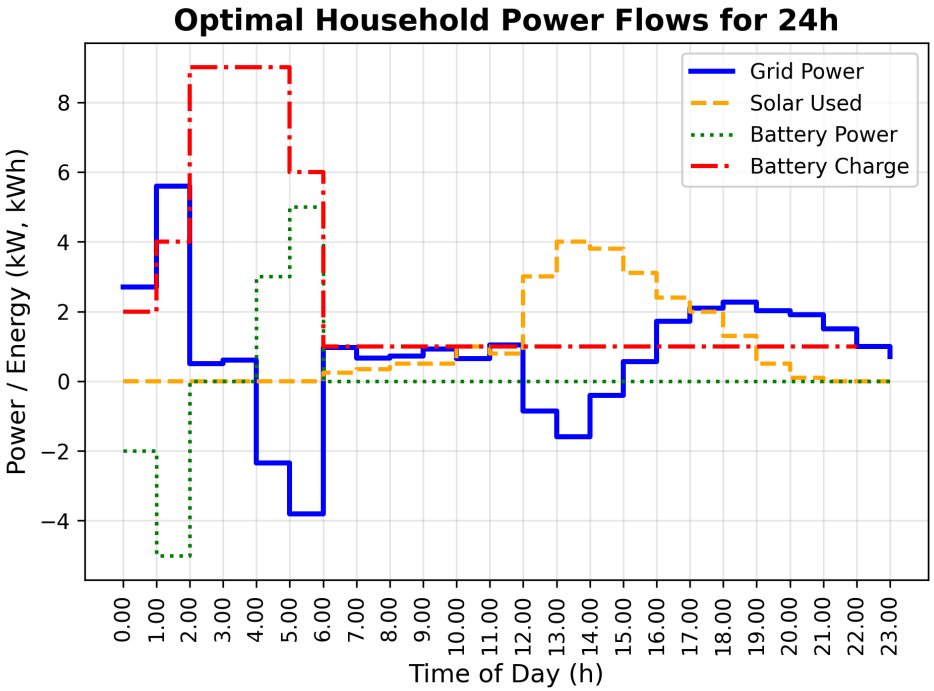


Figure 4: Buying and selling prices are both 0.20 USD/kWh. We can see some energy is sold to the grid in the afternoon, when there's energy surplus.

References

[1] "How Artificial Intelligence Enhances Hybrid Inverter Performance." [Online]. Available: https://www.sigenergy.com/en/news/info/1837.html

[2] "Daily Household Power Consumption." [Online]. Available: https://www.cromwellsolar.com/knowledge-center/time-of-use

[3] "PV production curves on a summer day and a winter day (only summer day used." [Online]. Available: https://www.researchgate.net/figure/Examples-for-generated-PV-production-curves-on-a-summer-day-and-a-winter-day_fig1_257380310